

Student Answer Sheet

Observability & Centralized Logging

(Chapter 5)

Course: Systems Reliability Engineering (SRE)

Lab: Observability & Centralized Logging

Student Name: ATHAMNI Amine
BELKERDID Islam
TALEB Mehdi
DJEZIRI Mohammed Yassine
BOUDIAF Soufiane

Group Number: 5

1 The Challenge (Section 2)

Explain why binary “up/down” monitoring failed in this instance. Why is monitoring “pre-defined questions” insufficient for identifying p99 latency spikes in a distributed microservices environment?

Binary “up/down” monitoring is fundamentally limited because it can only answer pre-defined questions such as “Is port 443 open?” or “Did the server return HTTP 200?”. These checks confirm the *existence* of a response, not the *quality* of the user experience.

In a distributed microservices environment, the checkout request passes through multiple independent services — authentication, inventory, payment gateway, and shipping calculation. Each service may individually report healthy status with HTTP 200 responses, yet the cumulative latency across all hops can produce extreme tail latency (p99) that directly impacts users.

Traditional monitoring cannot observe this because it was never asked the right question. p99 latency is an “unknown-unknown” — a failure mode that was never anticipated and therefore never instrumented. Only a fully observable system — one that records the internal state through logs, metrics, and distributed traces — can surface these silent failures. Observability allows engineers to ask *new* questions of the data after a problem is discovered, without having pre-instrumented for that exact failure.

2 Part 1: The Three Pillars (Section 3)

2.1 Pillar Characteristics Table

Complete the table below:

Three Pillars of Observability

Pillar Name	Data Characteristic (Discrete vs. Aggregated, Cardinality)	Specific Use Case (E-Commerce Scenario)
Logs	Discrete events, high cardinality, unstructured or semi-structured text. Each log is a timestamped record of one specific event.	Recording each checkout attempt: <code>user_id=5821</code> <code>action=payment_submit</code> <code>status=error</code> <code>timestamp=...</code> Used to reconstruct exactly what happened during a failed transaction.
Metrics	Discrete events, high cardinality, unstructured or semi-structured text. Each log is a timestamped record of one specific event.	Tracking requests-per-second, CPU%, memory usage, and — critically — p99 checkout response time over time. Metrics would show the latency spike on a graph.
Traces	Distributed request lifecycle tracking. One trace = one user request spanning multiple services. Contains “spans” for each service hop.	A single checkout trace showing: Auth=12 ms, Inventory=8 ms, Payment=2,400 ms, Shipping=9 ms. Immediately reveals the Payment service as the bottleneck even though it returned HTTP 200.

2.2 Diagnostic Question (Section 3.1)

How do **Traces** specifically resolve the problem of identifying bottlenecks in a distributed request lifecycle where every individual service appears healthy but the overall response is delayed?

Answer:

Distributed traces resolve the bottleneck problem by propagating a unique Trace ID through every service a request touches. Each service records a “span” — its own start time, end time, and status — and links it to the parent trace. When the spans are assembled in a tool like Kibana or Jaeger, the engineer sees the complete timeline of the request across all services simultaneously.

In our scenario, every individual service returned HTTP 200 (healthy), so metrics and basic health checks showed nothing wrong. However, a trace would reveal that while the Auth service took 12 ms and the Inventory service took 8 ms, the Payment service took 2,400 ms before returning its successful 200 response. This hidden latency is invisible to per-service monitoring but is immediately obvious in the aggregated trace timeline. The trace answers not just “did it work?” but “where did the time go?” — which is the precise question binary monitoring cannot ask.

3 Part 2: ELK Technical Analysis (Section 4)

3.1 Beats & Elastic Agent

Answer:

Beats are lightweight, single-purpose data shippers installed directly on edge nodes (servers, VMs, containers). They consume minimal resources critical for production servers that cannot afford overhead and stream telemetry to the central Logstash pipeline. For example, Filebeat tails log files and forwards new lines in near-real-time.

The management challenge with standalone Beats arises at scale: updating configurations on hundreds of servers individually is error-prone and slow. Elastic Agent solves this by providing a single unified agent that handles logs, metrics, and security data simultaneously. Managed via the Fleet UI in Kibana, configuration changes are pushed centrally to all enrolled agents simultaneously, dramatically reducing operational overhead and ensuring configuration consistency across the entire fleet without SSH-ing into individual machines.

3.2 Logstash & Grok

Answer:

Logstash is the processing engine of the ELK pipeline. It receives raw, unstructured log strings from Beats and applies Grok filters — a library of named regular expressions — to parse these strings into structured JSON fields. For example, a raw Nginx log line is a single opaque string, but after a Grok filter applying the `%{COMBINEDAPACHELOG}` pattern, it becomes a structured document with distinct fields for IP address, HTTP method, URI path, status code, and response size.

The enrichment phase extends this further. GeoIP enrichment resolves the client IP address to a physical location (city, country, coordinates), enabling geographic attack pattern analysis. Adding user-agent parsing identifies device types. This enrichment happens during ingestion — before storage — meaning every query run later automatically benefits from the enriched metadata without reprocessing. In an e-commerce context, this allows instant correlation between checkout failures and geographic regions or device types.

3.3 Elasticsearch & The Inverted Index

Answer:

Elasticsearch uses an inverted index architecture, borrowed from information retrieval systems. Instead of storing data in rows and scanning each row during a search (as a relational database would), it builds an index that maps every unique term to the list of documents containing it. When a query arrives for `status_code:500`, Elasticsearch looks up “500” in the index and instantly retrieves the pre-computed list of matching document IDs — regardless of whether the dataset contains 1 million or 100 billion log entries.

This is fundamentally superior for log analysis because logs are write-once, read-many

data with highly variable schemas and enormous scale. A relational database with a B-tree index would perform a sequential scan or require pre-known schema constraints that logs inherently lack. Elasticsearch's schema-on-read model allows ingesting any log format and querying arbitrary fields at terabyte scale with sub-second response times — making forensic investigations across months of historical data practically feasible.

3.4 Kibana

Answer:

Kibana serves as the unified analyst interface across all three observability pillars. Beyond static dashboards, its power lies in cross-pillar correlation. An engineer investigating a p99 latency spike can begin in the Metrics view — identifying the exact 3-minute window where checkout latency exceeded 2 seconds. With one click, Kibana's time filter is applied globally, and the Logs view instantly shows only the log entries from that window. From a log entry containing a Trace ID field, the engineer can navigate directly to the distributed Trace view to see the full request breakdown across all microservices.

Kibana's Lens and Canvas tools enable creating executive dashboards showing business KPIs alongside technical SLIs — for example, overlaying revenue-per-minute with p99 checkout latency to make the business impact of technical degradation immediately visible to non-engineering stakeholders.

4 Part 3: WEF Design Challenge (Section 5)

4.1 Architecture Evaluation Table

WEF Architecture Comparison

Feature	Source-Initiated (Push)	Collector-Initiated (Pull)
Scalability	Highly scalable. Each source manages its own connection. Adding new sources requires no reconfiguration of the collector.	Less scalable. Collector must maintain a list of all sources and poll each one. Adding sources requires updating the collector's subscription list.
Firewall Configuration	Simpler for sources behind NAT or strict firewalls. Sources open outbound connections only (port 5985). The collector needs one inbound port open.	Complex. The collector must be able to reach every source machine. Every source must allow inbound connections — difficult for machines behind firewalls or NAT.

Suitability for Remote Laptops	Ideal. Laptops can push logs whenever they connect to any network, including VPN. The collector doesn't need to know the laptop's current IP address.	Poorly suited. Laptops have dynamic IPs and are frequently offline or on external networks. The collector cannot reliably initiate connections to them.
---------------------------------------	---	---

4.2 Windows Server 2025 Innovation (Section 5.2.1)

Answer:

Traditional Kerberos authentication requires the authenticating machine to have direct network line-of-sight to a Domain Controller (DC) to obtain a Kerberos ticket. In remote scenarios — laptops on VPN, machines in isolated network segments, or field devices — this requirement forces a fallback to NTLM, a legacy protocol with known vulnerabilities including pass-the-hash attacks and lack of mutual authentication.

Windows Server 2025 introduces two innovations to address this: IAKerb (Initial and Pass-Through Authentication using Kerberos) allows a client to route Kerberos authentication requests through an intermediary server that does have DC connectivity, without requiring direct DC access itself. The Local KDC allows a machine to issue Kerberos tickets locally for local account authentication, eliminating the DC dependency entirely for local scenarios.

In a WEF context, this means that when a remote laptop establishes a WinRM connection to push its security logs to the WEF Collector, it can now authenticate using Kerberos instead of NTLM — even without VPN connectivity to the DC. This closes a significant security gap where NTLM was the only option, making the entire log collection channel more cryptographically secure and resistant to relay attacks.

4.3 Forensic Prioritization (Section 5.2.2)

Identify the two specific Windows Event IDs for logon success and failure:

Logon Success Event ID: 4624

Logon Failure Event ID: 4625

5 Part 4: SLIs and Error Budgets (Section 6)

5.1 Error Budget Calculation (Section 6.1)

Given: 99.95% SLO. Total minutes in 30 days = 43,200.

Show your work:

$$SLO = 99.95\%$$

$$\text{Total minutes in 30 days} = 43,200$$

$$\text{Error Budget} = \text{Total minutes} \times (1 - SLO)$$

$$\begin{aligned} &= 43,200 \times (1 - 0.9995) \\ &= 43,200 \times 0.0005 \\ &= \mathbf{21.6 \text{ minutes per month}} \end{aligned}$$

Final Answer: 21.6 minutes per month

5.2 SLI Proposal (Section 6.2)

Answer:

Proposed SLI: “The proportion of checkout requests that complete with an end-to-end response time of less than 800 ms, measured at the 99th percentile (p99) over a 5-minute rolling window.”

p99 latency is a superior SLI compared to mean (average) latency because averages are heavily influenced by the majority of fast requests and mathematically mask the experience of the slowest users. In our scenario, if 98% of checkouts complete in 100 ms but 2% take 5,000 ms, the mean might show a “healthy” 198 ms. However, the p99 would correctly surface the 5,000 ms degradation, directly correlating with the user complaints. Since the slowest users are often those most likely to abandon — and since high-value transactions can cluster in the tail — p99 latency is a direct proxy for user experience and revenue impact.

5.3 The “SRE Clamp” (Section 6.3)

Answer:

If the checkout slowness continues and exhausts the 21.6-minute monthly error budget, the SRE team must invoke the “error budget policy” — formally halting all non-critical feature releases and platform changes until the budget is replenished at the start of the next month, or until the reliability issue is resolved.

This mechanism, known as the SRE Clamp, aligns incentives between development velocity and system reliability. Feature development consumes risk capacity; when that capacity is exhausted, the engineering organization’s priority automatically shifts to stability work. This prevents the compounding pattern where new features are shipped into an already-degraded system, further eroding reliability. The clamp is not punitive — it is a pre-agreed contractual mechanism that removes the need for conflict between SRE and product teams during incidents.

6 Part 5: Practical Implementation (Section 7)

6.1 Task 5.1: docker-compose.yml

```
1 version: "3.8"
2 volumes:
3   esdata01:
4     driver: local
5   kibana01:
```

```
6     driver: local
7 logstashdata01:
8     driver: local
9
10 networks:
11     default:
12         name: elastic
13         external: false
14
15 services:
16     elasticsearch:
17         image: docker.elastic.co/elasticsearch/elasticsearch:${
18             STACK_VERSION}
19         container_name: elasticsearch
20         environment:
21             - discovery.type=single-node
22             - xpack.security.enabled=false
23             - cluster.name=${CLUSTER_NAME}
24             - "ES_JAVA_OPTS=-Xms${ES_MEM_LIMIT} -Xmx${ES_MEM_LIMIT}"
25         ports:
26             - "${ES_PORT}:9200"
27         volumes:
28             - esdata01:/usr/share/elasticsearch/data
29         healthcheck:
30             test: ["CMD-SHELL", "curl -s http://localhost:9200 | grep -
31                 q 'cluster_name'"]
32             interval: 10s
33             timeout: 10s
34             retries: 120
35
36 logstash:
37     image: docker.elastic.co/logstash/logstash:${STACK_VERSION}
38     container_name: logstash
39     ports:
40         - "5044:5044"
41     volumes:
42         - logstashdata01:/usr/share/logstash/data
43         - ./logstash.conf:/usr/share/logstash/pipeline/logstash.
44             conf
45     environment:
46         - xpack.monitoring.enabled=false
47     depends_on:
48         elasticsearch:
49             condition: service_healthy
50
51 kibana:
52     image: docker.elastic.co/kibana/kibana:${STACK_VERSION}
53     container_name: kibana
54     ports:
55         - "${KIBANA_PORT}:5601"
```

```
54 volumes:
55   - kibana_data:/usr/share/kibana/data
56 environment:
57   - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
58   - ELASTICSEARCH_USERNAME=kibana_system
59   - ELASTICSEARCH_PASSWORD=${KIBANA_PASSWORD}
60   - XPACK_SECURITY_ENCRYPTIONKEY=${ENCRYPTION_KEY}
61   - XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY=${
      ENCRYPTION_KEY}
62 depends_on:
63   elasticsearch:
64     condition: service_healthy
65
66 checkout-microservice:
67   build:
68     context: ./microservice
69     dockerfile: DockerFile
70   container_name: checkout-microservice
71   ports:
72     - "5000:5000"
73   networks:
74     - default
75   depends_on:
76     elasticsearch:
77       condition: service_healthy
78   logging:
79     driver: "json-file"
80     options:
81       max-size: "10m"
82       max-file: "3"
83 filebeat:
84   image: docker.elastic.co/beats/filebeat:${STACK_VERSION}
85   container_name: filebeat
86   user: root
87   command: filebeat -e --strict.perms=false
88   volumes:
89     - ./microservice/filebeat.yml:/usr/share/filebeat/
      filebeat.yml:ro
90     - /var/lib/docker/containers:/var/lib/docker/containers:
      ro
91     - /var/run/docker.sock:/var/run/docker.sock:ro
92   depends_on:
93     elasticsearch:
94       condition: service_healthy
95     logstash:
96       condition: service_started
97   networks:
98     - default
```

Listing 1: Listing 1: docker-compose.yml

6.2 Task 5.2: filebeat.yml nginx server

```

1  # INPUTS SECTION
2  # Tells Filebeat WHAT to read
3  filebeat.inputs:
4    - type: log                # Read from log files
5      enabled: true           # Turn this input on
6      paths:
7        - /var/log/nginx/access.log # Read THIS specific file
8      tags: ["nginx", "webserver"] # Label these logs for easy
                                   filtering in Kibana
9  # OUTPUT SECTION
10 # Tells Filebeat WHERE to send the logs
11 output.logstash:
12   hosts: ["192.168.230.1:5044"] # Send to Logstash on your
                                   Windows host
13                                   # 192.168.230.1 = your Windows
                                   PC
14                                   # 5044 = Logstash port
15 # LOGGING SECTION
16 # Tells Filebeat how to log its OWN activity
17 logging.level: info           # Log normal information
18 logging.to_files: true        # Save logs to files
19 logging.files:
20   path: /var/log/filebeat      # Where to save Filebeat's own logs
21   name: filebeat               # Log file name
22   keepfiles: 7                 # Keep 7 days of logs

```

Listing 2: Listing 2: filebeat.yml

6.3 Task 5.2: filebeat.yml microservice server

```

1  filebeat.inputs:
2    - type: container
3      enabled: true
4      paths:
5        - /var/lib/docker/containers/*/*.log
6      processors:
7        - add_docker_metadata:
8            host: "unix:///var/run/docker.sock"
9  processors:
10   - drop_event:
11       when:
12         not:
13           equals:
14             container.name: "checkout-microservice"
15  output.logstash:
16   hosts: ["logstash:5044"]
17  logging.level: info

```

Listing 3: Listing 2: filebeat.yml

6.4 Task 5.3: winlogbeat.yml

```
1 winlogbeat.event_logs:
2   - name: Security
3     event_id: 4624, 4625
4
5 output.logstash:
6   hosts: ["192.168.230.1:5044"]
7
8 logging.level: info
```

Listing 4: Listing 3: winlogbeat.yml

6.5 Task 5.4: logstash.conf

```
1 input {
2   beats {
3     port => 5044
4   }
5 }
6
7 filter {
8
9   # NGINX LOGS      from Ubuntu VM Filebeat
10  if "nginx" in [tags] or "nginx" in [source] or "access" in [log
11    ][file][path] {
12
13    grok {
14      match => { "message" => "(?:{%IP:client_ip}|{%IPV6:
15        client_ip}) {%DATA:ident} {%DATA:auth} \[%{HTTPDATE:
16        timestamp}\] \"{%WORD:http_method} {%DATA:request_path}
17        HTTP/{%NUMBER:http_version}\" {%NUMBER:status_code:int}
18        {%NUMBER:response_size:int} \"{%DATA:referrer}\" \"{%DATA
19        :user_agent}\""} }
20
21    }
22
23    date {
24      match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z" ]
25      target => "@timestamp"
26    }
27
28    mutate {
29      convert => {
30        "status_code" => "integer"
31        "response_size" => "integer"
32      }
33      add_field => {
34        "environment" => "lab"
35        "server" => "nginx"
36        "service_type" => "nginx"
37      }
38    }
39  }
40 }
```

```
32
33     if [status_code] >= 400 {
34         mutate { add_tag => ["error"] }
35     }
36
37     if [status_code] == 200 {
38         mutate { add_tag => ["success"] }
39     }
40 }
41
42 # MICROSERVICE LOGS      from Docker container Filebeat
43 else if [container][name] == "checkout-microservice" {
44
45     json {
46         source => "message"
47         target => "app"
48     }
49
50     mutate {
51         add_field => {
52             "service_type" => "microservice"
53             "environment" => "lab"
54         }
55         add_tag => ["microservice", "checkout"]
56     }
57
58     if [app][level] == "ERROR" {
59         mutate { add_tag => ["error"] }
60     }
61
62     if [app][level] == "WARNING" {
63         mutate { add_tag => ["warning"] }
64     }
65
66     if [app][level] == "INFO" {
67         mutate { add_tag => ["success"] }
68     }
69 }
70
71 # WINDOWS LOGS      from Winlogbeat
72 else {
73     mutate {
74         add_field => {
75             "service_type" => "windows"
76             "environment" => "lab"
77         }
78     }
79 }
80
81 }
```

```
83 output {
84
85     # NGINX      nginx index
86     if [service_type] == "nginx" {
87         elasticsearch {
88             hosts => ["http://elasticsearch:9200"]
89             index => "nginx-logs-%{+YYYY.MM.dd}"
90         }
91     }
92
93     # MICROSERVICE      microservice index
94     else if [service_type] == "microservice" {
95         elasticsearch {
96             hosts => ["http://elasticsearch:9200"]
97             index => "microservice-logs-%{+YYYY.MM.dd}"
98         }
99     }
100
101     # WINDOWS      windows index
102     else {
103         elasticsearch {
104             hosts => ["http://elasticsearch:9200"]
105             index => "windows-logs-%{+YYYY.MM.dd}"
106         }
107     }
108
109     stdout {
110         codec => rubydebug
111     }
112 }
```

Listing 5: Listing 4: logstash.conf

6.6 Task 5.5: Visual Proof

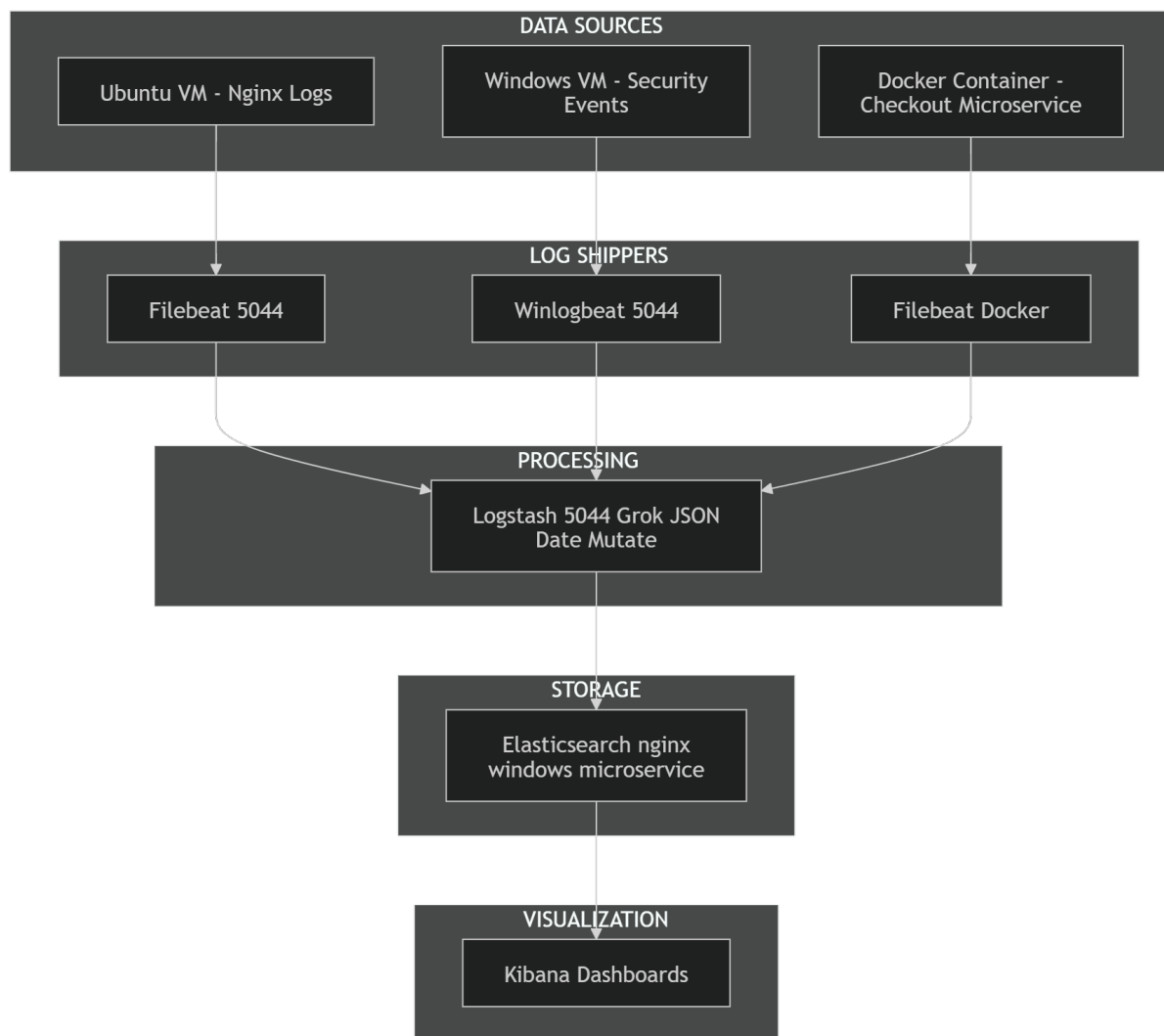


Figure 1: Architecture diagram

health	status	index	uuid	pri	rep	docs.count	docs.deleted	store.size	pri.store.size
yellow	open	windows-logs-2026.05.04	RmM_MvtpQRaNhynZLxcBjA	1	1	34	0	605.8kb	605.8kb
yellow	open	windows-logs-2026.05.05	8gxXtf1aQWYjkyq9dSA	1	1	104	0	638.1kb	638.1kb
yellow	open	microservice-logs-2026.05.05	PDXhynCYSf-Xjg9gIDPgMw	1	1	36	0	398.3kb	398.3kb
yellow	open	nginx-logs-2026.05.03	DGR730V8TnWQxUPAX-K-vQ	1	1	76	0	224kb	224kb
yellow	open	logs-2026.05.04	iUDQgOVVRASF-pJ4yTaMw	1	1	1	0	14.7kb	14.7kb
yellow	open	logs-2026.05.03	uS40xhDdRdqPnPxUuWOHKg	1	1	76	0	101.8kb	101.8kb
yellow	open	nginx-logs-2026.05.04	OQvaJr6NRlidxJerzZYS9g	1	1	175	0	504kb	504kb

Figure 2: elasticsearch indexes

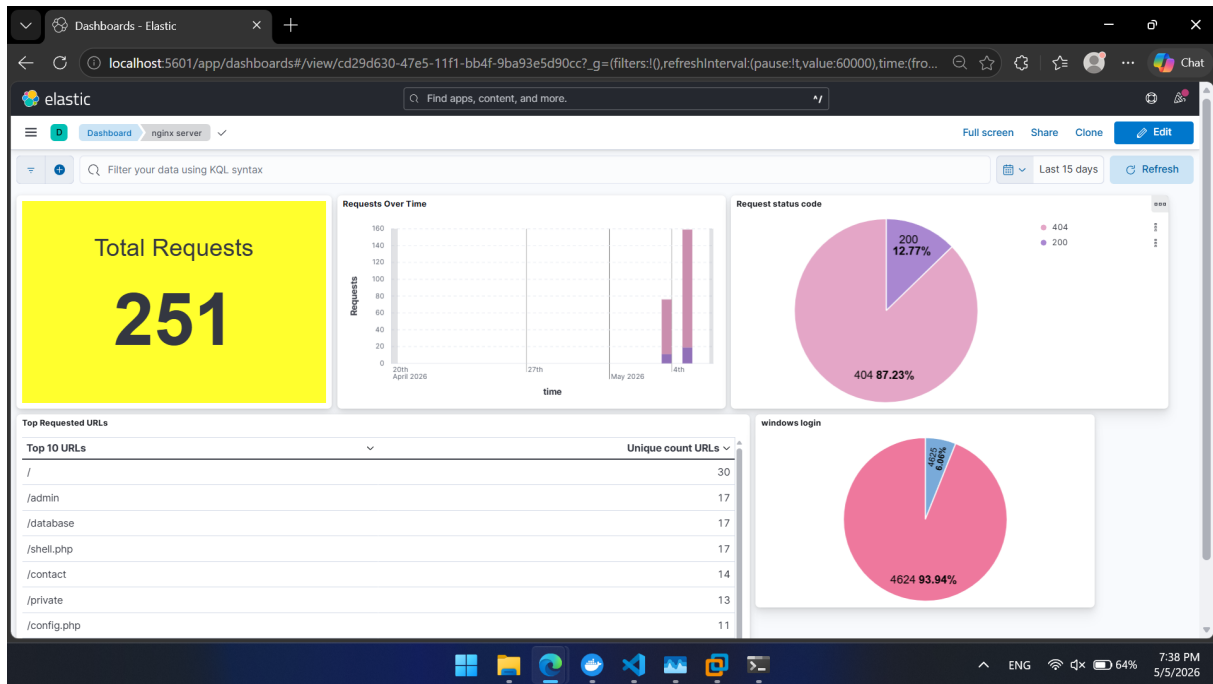


Figure 3: Server dashboard

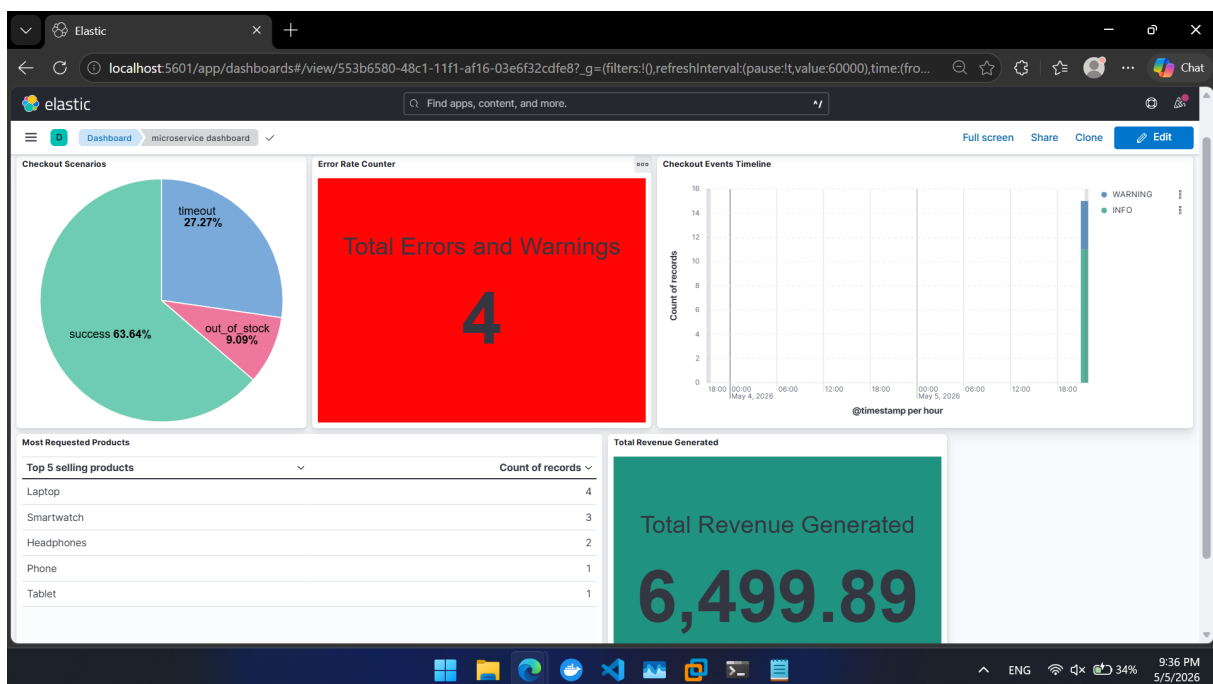


Figure 4: Microservice dashboard

7 Final Deliverable: System Interaction Map (Section 8)

Data Source 1

Source Name:	Ubuntu Linux Web Server (Nginx)
Telemetry Type: (Log / Metric / Trace)	Log
Shipper:	Filebeat 8.7.1 — installed directly on the Ubuntu VM, monitors <code>/var/log/nginx/access.log</code>
Identity Protocol/Tool:	TCP Beats Protocol over port 5044 — Filebeat opens an outbound TCP connection to Logstash on the host machine at <code>192.168.230.1:5044</code>
Processing / Transformation Step:	Logstash applies a Grok filter using a combined regex pattern to parse each raw Nginx log line into structured fields: <code>client_ip</code> , <code>http_method</code> , <code>request_path</code> , <code>status_code</code> , <code>response_size</code> , <code>user_agent</code> , and <code>timestamp</code> . A Date filter normalises the timestamp into ISO-8601 <code>@timestamp</code> . A Mutate filter adds enrichment fields (<code>environment=lab</code> , <code>server=nginx</code>) and applies <code>error</code> or <code>success</code> tags based on HTTP status code.
Final Destination / Storage:	Elasticsearch — daily rolling index <code>nginx-logs-YYYY.MM.dd</code> stored in the <code>esdata01</code> Docker volume
Visualization Tool:	Kibana — Nginx Monitor Dashboard featuring: HTTP Status Distribution Pie Chart (200 vs 404), Requests Over Time bar chart, Top Requested URLs table, and Total Requests metric counter

Data Source 2

Source Name:	Windows Server Virtual Machine (Security Event Log)
Telemetry Type: (Log / Metric / Trace)	Log
Shipper:	Winlogbeat 8.7.1 — installed as a Windows Service on the Windows VM, subscribed to the Security event channel filtering for Event IDs 4624 and 4625
Identity Protocol/Tool:	TCP Beats Protocol over port 5044 — Winlogbeat opens an outbound TCP connection to Logstash on the host machine at 192.168.230.1:5044
Processing / Transformation Step:	Logstash receives pre-structured Windows Event Log JSON documents from Winlogbeat. A Mutate filter adds the enrichment field <code>service_type=windows</code> and <code>environment=lab</code> . Event ID 4624 represents a Successful Logon and Event ID 4625 represents a Failed Logon — both are routed to the Windows index without further parsing as Winlogbeat already structures the fields.
Final Destination / Storage:	Elasticsearch — daily rolling index <code>windows-logs-YYYY.MM.dd</code> stored in the <code>esdata01</code> Docker volume
Visualization Tool:	Kibana — Windows Security Dashboard featuring: Security Events table showing Event IDs, Failed vs Successful Logon comparison chart, and a brute-force detection timeline showing repeated Event ID 4625 spikes

Data Source 3

Source Name:	Checkout Microservice (Containerised Python/Flask Application)
Telemetry Type: (Log / Metric / Trace)	Log
Shipper:	Filebeat 8.7.1 running as a Docker container on the host machine, using the <code>container</code> input type. It reads all container log files from <code>/var/lib/docker/containers/*//*.log</code> via the Docker socket and uses the <code>add_docker_metadata</code> processor to attach container name, image, and ID. A <code>drop_event</code> processor filters out all containers except <code>checkout-microservice</code> .
Identity Protocol/Tool:	TCP Beats Protocol over port 5044 — Filebeat communicates with Logstash using the internal Docker network name <code>logstash:5044</code> since both containers share the <code>elastic</code> Docker network
Processing / Transformation Step:	Logstash detects the <code>container.name=checkout-microservice</code> field and applies a JSON filter to parse the structured application log into the <code>app</code> sub-object, extracting fields including <code>app.level</code> , <code>app.scenario</code> , <code>app.user_id</code> , <code>app.product_name</code> , <code>app.price</code> , and <code>app.latency_ms</code> . A Mutate filter adds <code>service_type=microservice</code> and tags events as <code>error</code> , <code>warning</code> , or <code>success</code> based on log level.
Final Destination / Storage:	Elasticsearch — daily rolling index <code>microservice-logs-YYYY.MM.dd</code> stored in the <code>esdata01</code> Docker volume
Visualization Tool:	Kibana — Checkout Microservice Monitor Dashboard featuring: Checkout Scenario Distribution Pie Chart (success / timeout / payment_failed / out_of_stock), Checkout Events Timeline by log level, Total Revenue Sum metric, Most Requested Products table, and Total Errors and Warnings counter